Code Explanation

Data Extraction Module Explanation

This code is a data extraction module designed for E2E Policy Services. It utilizes PySpark to create a SparkSession, read data from source tables in a database, and extract the required data into DataFrames.

Overview of the Code

The code consists of three main functions: creating a SparkSession, reading a source table from the database, and extracting all required source tables. It relies on configuration settings from an external module to connect to the database and identify the source tables.

Step 1: Creating a SparkSession
The first step is to create a SparkSession, which is the entry point to any PySpark functionality. This is achieved through the `get_spark_session` function.

```
def get_spark_session() -> SparkSession:
    return (SparkSession.builder
            .appName("E2E_BC_K2H_DATA_EXTRACTION")
            .config("spark.sql.legacy.timeParserPolicy", "LEGACY")
            .config("spark.sql.sources.partitionOverwriteMode", "dyn
amic")
            .getOrCreate())
```

Step 2: Reading a Source Table
The `read_source_table` function reads a specified source table from the database using the provided SparkSession. It relies on the database configuration and source table names defined in an external configuration module.

```
def read_source_table(spark: SparkSession, table_name: str) -> DataF
rame:
    source_config = DB_CONFIG["source"]
    return (spark.read
            .format("jdbc")
            .option("url", source_config["jdbc_url"])
            .option("dbtable", SOURCE_TABLES[table_name])
            .option("user", source_config["user"])
            .option("password", source_config["password"])
            .option("driver", source_config["driver"])
            .load())
```

Step 3: Extracting All Required Source Tables
The `extract_source_data` function iterates over the list of required source tables, reads each one using the `read_source_table` function, and stores the resulting DataFrames in a dictionary.

```
def extract_source_data(spark: SparkSession) -> Dict[str, DataFrame]
:
    source_tables = {}
```

```
    for table_name in SOURCE_TABLES.keys():
        print(f"Extracting {table_name}...")
        source_tables[table_name] = read_source_table(spark, table_n
ame)
    return source_tables
```